

DOCUMENTO DE PROYECTO

"LO QUE QUIERAS!" - AGREGADOR INTELIGENTE DE MARKETPLACES

INFORMACIÓN GENERAL DEL PROYECTO

Campo	Valor
Nombre del Proyecto	Lo que quieras!
Versión del Documento	1.1
Fecha	Mayo 2025
Tipo de Proyecto	Aplicación Móvil con Agregación de Marketplaces
Estado	Diseño y Planificación

1. RESUMEN EJECUTIVO

"Lo que quieras!" es una aplicación móvil que agrega y centraliza productos de múltiples marketplaces mediante técnicas de web scraping y APIs oficiales, permitiendo a los usuarios buscar productos con filtros avanzados, comparación de precios en tiempo real y notificaciones inteligentes.

La aplicación busca resolver el problema de la fragmentación de información en marketplaces digitales y la dificultad de encontrar las mejores ofertas de manera eficiente, proporcionando una experiencia unificada y superior de búsqueda y comparación.

Propuesta de Valor

- Ahorro de tiempo:** Una sola búsqueda en múltiples plataformas
- Mejores precios:** Comparación automática entre marketplaces
- Alertas inteligentes:** Notificaciones cuando bajan los precios
- Experiencia superior:** Interfaz moderna con animaciones fluidas

2. ANÁLISIS DE MERCADO Y ESTRATEGIA

2.1 Problemática Identificada

- Los usuarios deben buscar manualmente en múltiples plataformas
- No existe comparación de precios automática entre marketplaces
- Información fragmentada y experiencias inconsistentes

- Pérdida de oportunidades por falta de alertas de precio

2.2 Marketplaces Objetivo (Estrategia Escalonada)

Fase 0: Investigación y Validación Legal

- **MercadoLibre** (API oficial disponible)
- **Amazon** (Programa de afiliados con API)
- **eBay** (API robusta y documentada)
- **OLX** (términos más permisivos)

Fase 1: MVP con Marketplaces Seguros

- Implementar APIs oficiales donde estén disponibles
- Scraping cuidadoso en plataformas que lo permitan
- Evitar Facebook Marketplace inicialmente

Fase 2: Expansión Controlada

- Evaluar viabilidad legal de Facebook Marketplace
- Agregar marketplaces locales
- Implementar sistema híbrido (APIs + scraping)

3. STACK TECNOLÓGICO DEFINITIVO

3.1 Frontend Mobile

javascript

```
{  
  "framework": "React Native + Expo SDK 51",  
  "styling": "NativeWind (TailwindCSS para RN)",  
  "navegacion": "React Navigation v6",  
  "estado": "Zustand + TanStack Query",  
  "formularios": "React Hook Form + Zod"  
}
```

3.2 Librerías de Animación (Estado del Arte 2025)

 **STACK COMPLETO DE ANIMACIONES:**

javascript

```
{
  "principal": "React Native Reanimated 3.6+",
  "gestos": "React Native Gesture Handler 2.14+",
  "texto": {
    "typewriter": "React Native Typewriter",
    "gradientes": "React Native Linear Gradient",
    "contador": "React Native Super Grid"
  },
  "micro-interacciones": "Lottie React Native 6.4+",
  "transiciones": "React Native Shared Element 2.0",
  "skeleton": "React Native Skeleton Placeholder",
  "parallax": "React Native Parallax Scroll View",
  "charts": "Victory Native XL (para gráficas de precios)"
}
```

3.3 Backend

javascript

```
{
  "runtime": "Node.js 20 LTS",
  "framework": "Express.js 4.19+ / Fastify 4.0+",
  "autenticacion": "JWT + Refresh Tokens",
  "validacion": "Zod",
  "rate-limiting": "express-rate-limit",
  "cache": "Redis 7.0+",
  "queue": "BullMQ (para jobs de scraping)",
  "monitoring": "Winston + New Relic"
}
```

3.4 Base de Datos

javascript

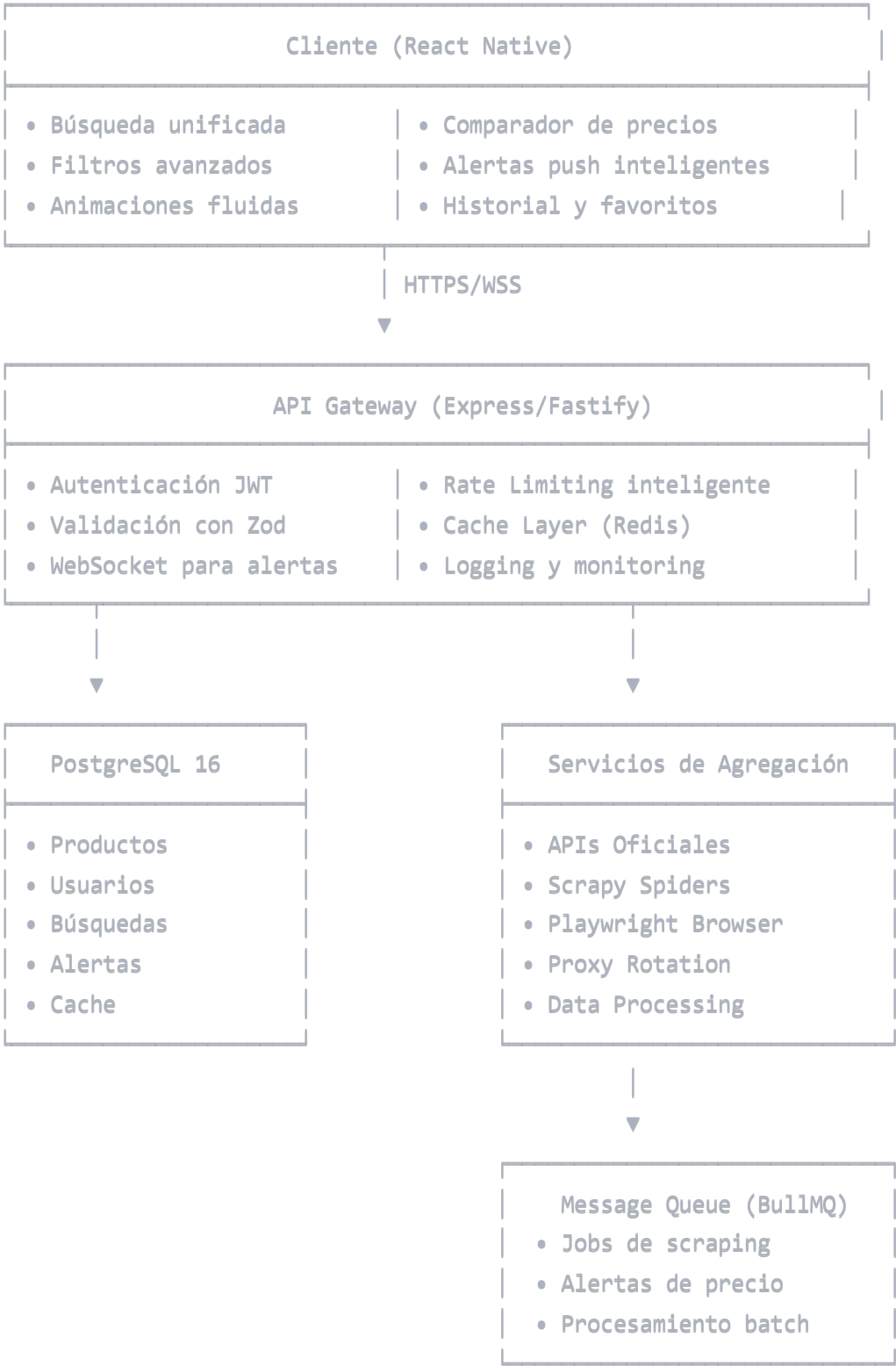
```
{
  "principal": "PostgreSQL 16",
  "ORM": "Prisma 5.0+",
  "cache": "Redis",
  "busqueda": "PostgreSQL Full Text Search + pg_trgm",
  "backup": "AWS RDS Automated Backups"
}
```

3.5 Servicios de Agregación

python

```
{  
  "lenguaje": "Python 3.12+",  
  "web_scraping": "Scrapy 2.11 + Playwright",  
  "api_clients": "httpx + aiohttp",  
  "parsing": "BeautifulSoup4 + lxml",  
  "proxy_rotation": "scrapy-proxy-middleware",  
  "anti_detection": "undetected-chromedriver",  
  "data_processing": "Pandas + NumPy"  
}
```

4. ARQUITECTURA DEL SISTEMA



5. ESPECIFICACIÓN FUNCIONAL

5.1 Funcionalidades Core (MVP)

Funcionalidad	Descripción	Prioridad	Estimación
Búsqueda Unificada	Búsqueda simultánea en múltiples marketplaces	Alta	2 semanas
Comparador de Precios	Vista comparativa con precios históricos	Alta	1.5 semanas
Filtros Avanzados	Precio, ubicación, categoría, condición, fecha	Alta	1 semana
Vista Detallada	Información completa + galería interactiva	Alta	1 semana
Cache Inteligente	Resultados optimizados por 30-60 minutos	Alta	0.5 semanas
Deep Linking	Redirección a marketplace original	Alta	0.5 semanas

5.2 Funcionalidades Avanzadas (v2.0)

Funcionalidad	Descripción	Prioridad	Estimación
Alertas Inteligentes	ML para detectar mejores momentos de compra	Media	2 semanas
Análisis de Precios	Gráficas de tendencias y predicciones	Media	1.5 semanas
Recomendaciones	Sistema ML de productos similares	Media	2 semanas
Chat Agregado	Comunicación unificada con vendedores	Baja	3 semanas
Marketplace Reviews	Sistema de reviews agregado	Baja	2 semanas

6. MODELO DE DATOS DETALLADO

6.1 Esquema de Base de Datos

sql

-- *Usuarios*

```
CREATE TABLE users (  
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    full_name VARCHAR(255),  
    phone VARCHAR(20),  
    preferences JSONB DEFAULT '{}',  
    subscription_tier VARCHAR(20) DEFAULT 'free',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- *Marketplaces*

```
CREATE TABLE marketplaces (  
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    domain VARCHAR(255) NOT NULL,  
    api_type VARCHAR(50), -- 'official_api', 'scraping', 'hybrid'  
    is_active BOOLEAN DEFAULT true,  
    rate_limit_per_minute INTEGER DEFAULT 60,  
    config JSONB DEFAULT '{}',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- *Productos*

```
CREATE TABLE products (  
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
    marketplace_id UUID REFERENCES marketplaces(id),  
    external_id VARCHAR(255) NOT NULL,  
    title VARCHAR(500) NOT NULL,  
    description TEXT,  
    price DECIMAL(12,2),  
    original_price DECIMAL(12,2),  
    currency VARCHAR(3) DEFAULT 'USD',  
    condition VARCHAR(50),  
    category VARCHAR(255),  
    subcategory VARCHAR(255),  
    location JSONB,  
    seller_info JSONB,  
    images JSONB,  
    specifications JSONB,  
    marketplace_url TEXT NOT NULL,
```



```

    is_available BOOLEAN DEFAULT true,
    stock_quantity INTEGER,
    shipping_info JSONB,
    first_seen_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_checked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(marketplace_id, external_id)
);

-- Historial de precios
CREATE TABLE price_history (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    product_id UUID REFERENCES products(id) ON DELETE CASCADE,
    price DECIMAL(12,2) NOT NULL,
    currency VARCHAR(3) DEFAULT 'USD',
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Búsquedas
CREATE TABLE searches (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    query VARCHAR(500) NOT NULL,
    filters JSONB DEFAULT '{}',
    results_count INTEGER DEFAULT 0,
    execution_time_ms INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Favoritos
CREATE TABLE favorites (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    product_id UUID REFERENCES products(id) ON DELETE CASCADE,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(user_id, product_id)
);

-- Alertas de precio
CREATE TABLE price_alerts (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    product_id UUID REFERENCES products(id) ON DELETE CASCADE,

```

```
target_price DECIMAL(12,2),
percentage_drop DECIMAL(5,2),
is_active BOOLEAN DEFAULT true,
notification_sent BOOLEAN DEFAULT false,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
triggered_at TIMESTAMP
);
```

-- Cache de búsquedas

```
CREATE TABLE search_cache (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  cache_key VARCHAR(64) UNIQUE NOT NULL,
  query VARCHAR(500),
  filters JSONB,
  results JSONB,
  hit_count INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  expires_at TIMESTAMP NOT NULL
);
```

-- Índices optimizados

```
CREATE INDEX idx_products_title_trgm ON products USING gin(title gin_trgm_ops);
CREATE INDEX idx_products_price ON products(price);
CREATE INDEX idx_products_location ON products USING gin(location);
CREATE INDEX idx_products_category ON products(category);
CREATE INDEX idx_products_updated ON products(last_updated_at DESC);
CREATE INDEX idx_price_history_product_date ON price_history(product_id, recorded_at DESC);
```

7. ESPECIFICACIONES DE API

7.1 Endpoints Principales

javascript

// Autenticación

POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh
POST /api/auth/logout
PUT /api/auth/profile

// Búsqueda y Productos

GET /api/search?q={query}&filters={filters}&page={page}
GET /api/products/:id
GET /api/products/:id/price-history
GET /api/products/trending
GET /api/products/recommendations/:userId
POST /api/products/compare

// Favoritos y Alertas

GET /api/favorites
POST /api/favorites
DELETE /api/favorites/:id
GET /api/alerts
POST /api/alerts
PUT /api/alerts/:id
DELETE /api/alerts/:id

// Marketplaces

GET /api/marketplaces
GET /api/marketplaces/:id/status

// Analytics

GET /api/analytics/search-trends
GET /api/analytics/price-trends/:category

// Admin

GET /api/admin/stats
POST /api/admin/scraping/trigger
GET /api/admin/system/health

7.2 Ejemplo de Respuesta de Búsqueda

javascript

```
{
  "query": "iphone 15",
  "filters": {
    "priceMin": 500,
    "priceMax": 1200,
    "location": "Mexico",
    "condition": "new"
  },
  "results": {
    "total": 156,
    "page": 1,
    "limit": 20,
    "products": [
      {
        "id": "uuid-here",
        "title": "iPhone 15 Pro 128GB Azul Titanio",
        "price": 999.99,
        "originalPrice": 1199.99,
        "currency": "USD",
        "marketplace": {
          "name": "MercadoLibre",
          "logo": "url-to-logo"
        },
        "condition": "new",
        "location": {
          "city": "Mexico City",
          "state": "CDMX"
        },
        "images": ["url1", "url2"],
        "priceHistory": {
          "lowest7Days": 950.00,
          "trend": "decreasing"
        },
        "seller": {
          "name": "TechStore MX",
          "rating": 4.8,
          "reviews": 1250
        },
        "shipping": {
          "free": true,
          "estimatedDays": "2-3"
        },
        "marketplaceUrl": "direct-link"
      }
    ]
  }
}
```

```
    }
  ]
},
"aggregations": {
  "priceRanges": {...},
  "marketplaces": {...},
  "conditions": {...}
},
"searchTime": 1.2,
"cached": false
}
```

8. PLAN DE DESARROLLO

8.1 Cronograma General

Fase	Duración	Descripción
Fase 0: Investigación	2 semanas	Análisis legal, APIs disponibles, proof of concept
Fase 1: MVP	8-10 semanas	Funcionalidades core con 2-3 marketplaces
Fase 2: Beta	4-6 semanas	Testing, optimización, feedback
Fase 3: Lanzamiento	2 semanas	Deploy, marketing, soporte
Fase 4: v2.0	8-12 semanas	Funcionalidades avanzadas

8.2 MVP Detallado (8-10 semanas)

Semanas 1-2: Configuración e Infraestructura

- Setup del repositorio y CI/CD
- Configuración de bases de datos
- Arquitectura base del backend
- Configuración de Redis y queues

Semanas 3-4: Backend Core

- Sistema de autenticación
- APIs básicas (usuarios, productos)
- Implementación de cache
- Setup de logging y monitoring

Semanas 5-6: Servicios de Agregación

- Integración con APIs oficiales (MercadoLibre, eBay)
- Sistema de scraping básico (OLX)
- Pipeline de procesamiento de datos
- Sistema de queues para jobs

Semanas 7-8: Frontend Mobile

- Configuración React Native + Expo
- Pantallas principales (búsqueda, resultados, detalle)
- Integración con APIs
- Implementación de animaciones básicas

Semanas 9-10: Integración y Testing

- Testing end-to-end
- Optimización de performance
- Bug fixing
- Preparación para beta

9. CONSIDERACIONES TÉCNICAS Y RIESGOS

9.1 Riesgos Técnicos

Riesgo	Probabilidad	Impacto	Mitigación
Bloqueo de scraping	Alta	Alto	APIs oficiales, proxies rotativos, rate limiting
Cambios en estructura web	Media	Medio	Detectores automáticos, alerts, fallbacks
Escalabilidad de base de datos	Baja	Alto	Sharding, read replicas, cache agresivo
Rate limiting de APIs	Media	Medio	Queue system, distribución de requests

9.2 Riesgos Legales

Riesgo	Mitigación
Violación de ToS	Enfoque en APIs oficiales, consulta legal
Copyright de imágenes	Solo thumbnails, link a original
Datos personales	GDPR compliance, minimización de datos

9.3 Estrategias de Mitigación

Sistema Anti-Bloqueo:

python

Configuración para scraping resiliente

```
SCRAPING_CONFIG = {  
    "user_agents": ["rotation_list"],  
    "proxy_rotation": True,  
    "request_delays": (1, 3), # seconds  
    "retry_attempts": 3,  
    "captcha_solver": "2captcha",  
    "browser_stealth": True  
}
```

Cache Inteligente:

- Cache L1: Redis (resultados frecuentes)
 - Cache L2: PostgreSQL (búsquedas históricas)
 - TTL dinámico basado en popularidad del producto
-

10. MODELO DE NEGOCIO Y MONETIZACIÓN

10.1 Estrategia de Monetización

Modelo Freemium:

- **Gratis:** 50 búsquedas/día, 5 alertas
- **Premium (\$9.99/mes):** Búsquedas ilimitadas, alertas ilimitadas, análisis avanzados
- **Pro (\$19.99/mes):** API access, datos en tiempo real, soporte prioritario

Fuentes de Ingresos Adicionales:

- **Comisiones de afiliados:** 2-5% en ventas completadas
- **API para desarrolladores:** \$0.01 por request
- **Anuncios premium:** Productos destacados de vendedores
- **Data insights:** Reportes de tendencias para empresas

10.2 Proyección Financiera (Año 1)

Métrica	Mes 3	Mes 6	Mes 12
Usuarios activos	1,000	10,000	50,000
Usuarios premium	50	800	5,000
Ingresos mensuales	\$500	\$8,000	\$45,000
Costos operativos	\$1,200	\$3,500	\$12,000

11. PLAN DE MARKETING Y LANZAMIENTO

11.1 Estrategia de Lanzamiento

Pre-lanzamiento (4 semanas):

- Landing page con lista de espera
- Content marketing (blog, redes sociales)
- Beta testers recruitment
- Partnerships con influencers de tech

Lanzamiento (2 semanas):

- Product Hunt launch
- Press kit para medios tech
- Campaña en redes sociales
- Programa de referidos

Post-lanzamiento (8 semanas):

- User feedback collection
- Iteración rápida basada en feedback
- Expansion a nuevos marketplaces
- Programa de embajadores

11.2 Canales de Adquisición

1. **Orgánico:** SEO, content marketing
2. **Social:** Instagram, TikTok, Twitter
3. **Partnerships:** Influencers, blogs de tech
4. **Paid:** Google Ads, Facebook Ads (ROI > 3:1)
5. **Referidos:** Programa de incentivos

12. MÉTRICAS Y KPIs

12.1 Métricas de Producto

Métrica	Objetivo	Frecuencia
DAU/MAU	>0.25	Diario
Retention Day 7	>40%	Semanal
Búsquedas por usuario	>3/sesión	Diario
Conversión a Premium	>8%	Mensual
NPS	>50	Trimestral

12.2 Métricas Técnicas

Métrica	Objetivo	Frecuencia
Tiempo de respuesta API	<500ms	Tiempo real
Uptime	>99.5%	Tiempo real
Error rate	<1%	Diario
Cache hit rate	>80%	Diario
Scraping success rate	>95%	Diario

13. EQUIPO Y RECURSOS NECESARIOS

13.1 Equipo Mínimo (MVP)

Rol	Dedicación	Skills requeridos
Full-stack Developer	100%	React Native, Node.js, PostgreSQL
Backend Developer	100%	Python, Scrapy, APIs
Product Owner	50%	UX/UI, Product Management
DevOps/QA	25%	AWS, CI/CD, Testing

13.2 Recursos Técnicos

Infraestructura Inicial:

- **Hosting:** AWS/DigitalOcean (~\$200/mes)
- **Base de datos:** PostgreSQL managed (~\$100/mes)

- **Cache:** Redis managed (~\$50/mes)
 - **CDN:** CloudFlare (~\$20/mes)
 - **Monitoring:** New Relic (~\$100/mes)
 - **Total:** ~\$470/mes
-

14. PRÓXIMOS PASOS INMEDIATOS

14.1 Semana 1-2: Validación Legal y Técnica

Tareas Críticas:

1. **Consulta legal especializada** en scraping y términos de servicio
2. **Análisis detallado de APIs disponibles:**
 - MercadoLibre API (documentación completa)
 - Amazon Product Advertising API
 - eBay Browse API
 - Términos de scraping de OLX
3. **Proof of concept técnico:**
 - Setup básico de scraping con Scrapy
 - Integración con una API oficial
 - Base de datos con datos de ejemplo

14.2 Decisiones Importantes a Tomar

1. **¿Empezar solo con APIs oficiales?** (Recomendado)
2. **¿Incluir Facebook Marketplace desde el inicio?** (No recomendado)
3. **¿Modelo de suscripción desde MVP?** (Sí, pero con plan gratuito generoso)
4. **¿Enfoque geográfico inicial?** (México/LATAM vs Global)

14.3 Documento de Decisiones

RECOMENDACIONES FINALES:

- ✅ **PROCEDER CON EL PROYECTO** - La idea es sólida y viable
- ✅ **EMPEZAR CON MARKETPLACES SEGUROS** - APIs oficiales primero
- ✅ **ARCHITECTURE MODERNA** - Stack tecnológico propuesto es excelente

⚠️ **VALIDACIÓN LEGAL CRÍTICA** - Antes de escribir código

⚠️ **MVP CONSERVADOR** - 2-3 marketplaces máximo inicialmente

⚠️ **PLAN B DEFINIDO** - Si scraping se complica, pivotar a APIs

15. CONCLUSIONES

"Lo que quieras!" tiene **excelente potencial** para convertirse en una aplicación exitosa. La propuesta técnica es sólida, el problema a resolver es real, y existe un mercado claro para la solución.

Factores de Éxito Clave:

- Ejecución técnica de calidad
- Experiencia de usuario superior
- Estrategia legal sólida
- Monetización equilibrada
- Crecimiento orgánico sostenible

El proyecto es VIABLE y RECOMENDABLE siguiendo las modificaciones y precauciones mencionadas en este documento.

Documento preparado por Claude Sonnet 4 | Mayo 2025